

Benchmarking Time Series Databases with IoTDB-Benchmark for IoT Scenarios

Rui Liu, Jun Yuan, Xiangdong Huang
Tsinghua University
Beijing, China

245422695@qq.com, richard_yuan16@163.com, huangxdong@tsinghua.edu.cn

ABSTRACT

With the wide application of time series databases (TSDB) in big data fields like cluster monitoring and industrial IoT, there have been developed a number of TSDBs for time series data management. Different TSDBs have test reports comparing themselves with other databases to show their advantages, but the comparisons are typically based on their own tools without using a common well-recognized test framework. To the best of our knowledge, there is no mature TSDB benchmark either. With the goal of establishing a standard of evaluating TSDB systems, we present the *IoTDB-Benchmark* framework, specifically designed for TSDB and IoT application scenarios. We pay close attention to some special data ingestion scenarios and summarize 10 basic queries types. We use this benchmark to compare five TSDB systems: *IoTDB*, *InfluxDB*, *OpenTSDB*, *KairosDB* and *TimescaleDB*. Our benchmark framework/tool not only measures *performance metrics* but also takes *system resource consumption* into consideration.

Keywords

benchmark, time series database, performance evaluation

1 Introduction

With the pervasive application of time series databases (TSDB) in big data fields, such as industrial IoT [25], manufacturing and power net, various time series databases have sprung up, including InfluxDB [22], KairosDB [14], TimescaleDB [24] and OpenTSDB [23]. In the space of such a large variety of TSDBs, how to choose the most appropriate database service that suits the business needs becomes an important issue for developers and IT managers. Therefore, a flexible benchmark tool that can effectively assess and compare the performance of different TSDBs is desired.

Traditional database benchmark, like the TPC-benchmark family [5], provides a wide range of benchmarks customized for specific application scenarios along with corresponding

official benchmark tools. However, there is currently no widely recognized nor well-designed benchmark or tool on the market particularly for TSDB. Besides, most existing benchmark tools do not support the management of configuration parameters or system resource monitoring data during the tests, let alone persist or help analyze test results.

TPCx-IoT [6] in TPC-benchmark family claims to be the first industry benchmark for time series oriented systems. It aims at comparing different software and hardware solutions for IoT gateways. However, the scenarios are not suitable for many practical use, including batch out-of-order data ingestion, aggregation or down-sampling querying.

Although a few time series based benchmark tools have been developed recently, most of them lack the ability to simulate diverse workloads of time series oriented applications. For example, TSDBBench uses YCSB-TS [7], an extended version of YCSB [12], as its official benchmark tool. Because YCSB is designed specifically for NoSQLs, the tool is hard to simulate flexible time series workloads, such as out-of-order data ingestion, time series with irregular frequency, and different data distributions. Other tools, such as InfluxDB-comparison [2], have similar drawbacks. All of these motivate us to establish a benchmark and tool to evaluate individual TSDB systems.

In this paper, we present IoTDB-Benchmark (<https://github.com/thulab/iotdb-benchmark>) that is specifically designed for time series databases. First, the benchmark requires specifying various data distribution, because many TSDBs apply different data encoding algorithms, such as RLE [21] and Gorilla [20], which have significantly different effects on different data distributions. Another reason is that data distribution varies significantly in different applications and benchmark should cover these different data distribution types. Second, the benchmark requires specifying two kinds of operations for data ingestion: out-of time order data ingestion or data in the time order for ingestion in batch. The two operations are common in time series oriented applications while most existing benchmarks are not considered both. Third, the benchmark requires specifying not only the data ingestion operations and the query operations, but also what operation system metrics need to be collected.

We designed a IoTDB-Benchmark tool to support the above features. First of all, the benchmark tool has a data generator for simulating *various data distributions*, including but not limited to square wave, sine wave and sawtooth wave with controllable Gaussian noise. Second, the data

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WOODSTOCK '97 El Paso, Texas USA

© 2024 ACM. ISBN 123-4567-24-567/08/06.

DOI: 10.475/123-4

generator can generate either [redacted] Third, the tool can be configured for not only individual basic operations, such as ingestion and query, but also [redacted] to simulate complex real world workloads. Fourth, the system [redacted], for example, CPU usage and memory usage, are recorded for analysis. Last but not the least, the tool provides a general approach to manage benchmark result data, including persisting test configuration and test data and analyzing test results. Using all the data recorded with the tool, we demonstrate the performance comparison of four TSDB systems: InfluxDB, KairosDB, OpenTSDB, and TimescaleDB under various workloads.

The organization of this paper is as follow, Section 2 depicts the typical scenarios in TSDB application. Section 3 discusses the fundamental workloads of our benchmark. Section 4 discusses what and how to measure the performance of a TSDB system. Section 5 introduces more details of our benchmark tool. Experiments in Section 6 compare performance of multiple TSDB systems. Section 7 examines related work, and Section 8 is our conclusions.

2 Scenarios

Different from many data center monitoring applications in IT companies, we mainly focus on industrial scenarios, which have more complex workloads. For example, a wind power company operates several wind-farms, each of which has some wind turbines and there are many sensors on each turbine for measuring hundreds of working metrics. Some sensors, such as temperature, generate periodical data with burrs. Some other sensors generate on-off value data or even data without obvious rules. Normally, the number of time series is large. For example, a wind power company in China operates 300 wind farms with total of 30 thousand wind turbines and each turbine collects at least 100 working metrics every 5 seconds or 7 seconds. Therefore, there are 3 million ($30,000 \times 100$) time series in total for all the turbines in this company.

Unlike many DevOps applications collecting data every minute, the sensors data are collected and sent to the data center in large variety of frequencies. Some sensors send data with a high frequency, e.g., 1000Hz. Some sensors apply several frequencies and switch the frequency at different work situations. Other sensors have even irregular frequencies.

In most cases, the data of sensors in a device are sent in batch (namely a packet) to the data center for reducing unnecessary transmission overhead. Then the server who receives packets may use one *batch insert* operation of database to write several packets to obtain a better throughput.

Although the time series data is generated in the time order, the timestamps of time series data sent to TSDB are likely out-of-order due to the network latency and asynchronous transmission operations. This means that timestamps of data are not strictly incremented when the data arrives time series database service.

Besides the ingestion workload to TSDB, another part of the workload is the queries. First, the most common case is that the user wants to query the data of a certain time range. Second, user wants to know the average or maximum value of the data. Third, the user may query the data within a certain value range. Besides, the data may be stored every millisecond but user only cares about data in seconds,

which needs sampling query to access data in different time granularity. Last but not least, the combination of all these requirements should also be considered.

To this end, we will examine the existing time series data test scenarios in further details and design a benchmark test configuration and test framework for industrial application scenarios.

3 Benchmark Workloads

In this section, we describe the general idea and principles that IoTDB-Benchmark should consider. For the scenarios described in Section 2, we present a flexible and scalable benchmark workload generator that can adjust the workload for a wide range of TSDB applications. We will explain the main parameters that users can configure for data ingestion and data query.

3.1 Data Ingestion

IoTDB-Benchmark allows several parameters to configure different workloads for data ingestion. Users can define: (1) the schema of time series; (2) the data distribution; (3) the behavior of the batch operation; (4) the timestamp distribution (i.e., the frequency); and (5) whether the data is ordered by the timestamp.

The schema of time series: we use *device group*, *device* and *sensor* to represent a time series. Users can define the number of device groups, the number of devices in each group and the number of sensors on each device. For each time series, users can define its value type, e.g., float, double, integer or string.

The data distribution: to better simulate the data in the real world, the benchmark defines the data generator to fit the periodic signal of the real scene according to some parameters. The generator supports five data distribution types for individual time series data: square wave, sine wave, sawtooth, random value within a certain range and constant value. In the first three types of data distribution, user can specify whether noise is added in the time series. Fig. 1 illustrates examples of the five types with and without noise.

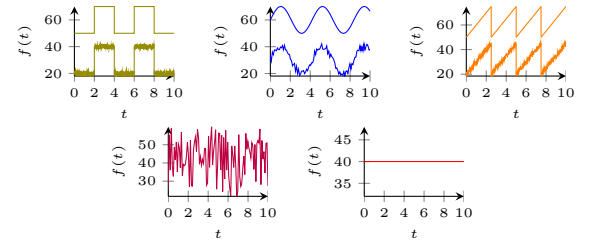


Figure 1: Five Types of Series Data Distribution

Batch operation: as described in Section 2, a TSDB usually receives data in batch. While each packet contains the data of all sensors in a device at certain timestamp, user can define how many packets are batch inserted in the target TSDB.

The timestamp distribution: by default, each time series generates data in a fixed frequency. However, the benchmark also allows user to specify whether the frequency is irregular.

Out-of-order data: there needs to have configuration parameters to control whether the timestamp is out of order

and the proportion of out-of-order data. There are three different types of out-of-order data to consider: (1) Batch insert out-of-order: the data within each batch is out of order, but different batches are in order, that is, the timestamp of the next batch is always greater than the timestamp of the previous batch; (2) Global out-of-order: batches are not guaranteed to be ordered along time, that is, the overall data input is out-of-order; (3) Poisson-distribution out-of-order: the timestamp of the time series data is generated by following a certain probability distribution. The detail of this mode is introduced in Section 5.3.

3.2 Query

IoTDB-Benchmark supports ten query types. Suppose there is a relational table `data` (`device`, `time`, `v1`, `v2`, ..., `vn`) (`vi` is short for the value of sensor *i*) for storing time series data, then the 10 queries can be described as:

- Q1-Exact point query, i.e., `select v1... from data where time=? and device in ?`.
- Q2-Time range query, i.e., `select v1... from data where time > ? and time < ? and device in ?`.
- Q3-Query with limit and without filters, i.e., `select v1... from data limit ?`.
- Q4-Time range query with value filter, i.e., `select v1... from data where time > ? and time < ? and v1 op ? and device in ?`.
- Q5-Q4 with clause `limit ?`.
- Q6-Aggregation query with time filter, i.e., `select func(v1)... from data where device in ? and time > ? and time < ?`.
- Q7-Aggregation query with value filter, i.e., `select func(v1)... from data where device in ? and value op ?`, where *op* represents `>`, `<` or `=`.
- Q8-Aggregation query with value filter and time filter, which is the combination of Q6 and Q7.
- Q9-Latest point query, i.e., `select time, v1... where device = ? and time = max(time)`.
- Q10-Group by time range query. Group by time range is hard to be represented by a standard SQL, but is useful for time series data, e.g., achieving down sampling. Suppose there is a time series which covers the data in 1 day. By grouping the data by 1 hour, we can get a new time series which only contains 24 data points. There are more examples in Section 5.4.

In the above queries, `func()` represents an aggregation function, such as `avg`, `min`, etc., and `...` represents that there is more than one column in the `select` clause.

4 Performance Metric

The performance of TSDB is evaluated by a set of metrics. First, a set of statistical metrics is needed to evaluate the performance of each type of operations, including minimum, maximum, average, middle-average, 1st, 5th, 50th, 90th, 95th and 99th percentile of **cost-time**. Cost-time is used as the performance measurement and it means the elapse time

between sending a request or statement to the TSDB and receiving the full result from the TSDB successfully, which is also called latency or TTLB (Time to Last Byte). Middle-average is the average cost-time that cuts off 5% head and tail.

Second, we use **throughput** to evaluate the performance of ingestion test, which is calculated by the cost-time and the number of concurrent clients. We add up the ingestion cost-time of each client, respectively, as accumulative cost-time for each client and take the maximum accumulative cost-time as the total cost-time of multiple concurrent ingestion clients. The throughput equals to the total number of ingested data points divided by the total cost-time.

Third, the used disk space of the TSDB system during the test process (mainly in ingestion test) is monitored. We took the maximum difference between the start space consumption and used disk space during the test process as the **space consumption** of the TSDB. That is, the space consumption may include the file of Write Ahead Log (WAL) [17], which is necessary for data recovery. Though the compression ratio of WAL files is smaller than the ratio of data files in many TSDBs, we consider the size of WAL is needed to be considered because it impacts how much total disk space an application needs.

Fourth, the **system resources consumption** during the test process is also considered. We measure several important system resource metrics including the system CPU, memory and disk I/O usage, memory used by TSDB service process, network receiving rate, disk I/O transfer number per second (tpc), disk write/read speed, etc.

5 Benchmark Tool

5.1 The Process

Fig. 2 shows the benchmark test process and other extensions that our benchmark tool supports. As shown in Fig. 2, the whole test process can be divided into 6 phases, the box of solid line represents standard procedure of benchmark framework and the box in dotted line extended feature of the tool.

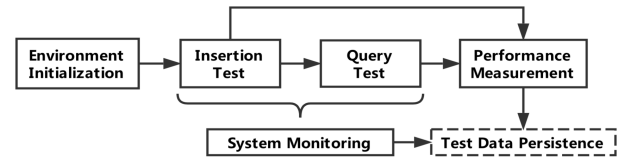


Figure 2: Benchmark Test Procedures and Features Supported by the Benchmark Tool

5.2 The architecture

Environment initialization. The first step of conducting a new test is to set up the test environment. In this step, users just need to configure parameters to define a workload, then our benchmark tool is able to automatically initialize the test environment, including starting the target TSDB service, removing old data and creating data schema if necessary. This initial setup is only executed before ingestion test. In another word, ingestion test starts with a cleaned, empty TSDB system. Besides, the old data cleanup process can be enabled or disabled through configuration.

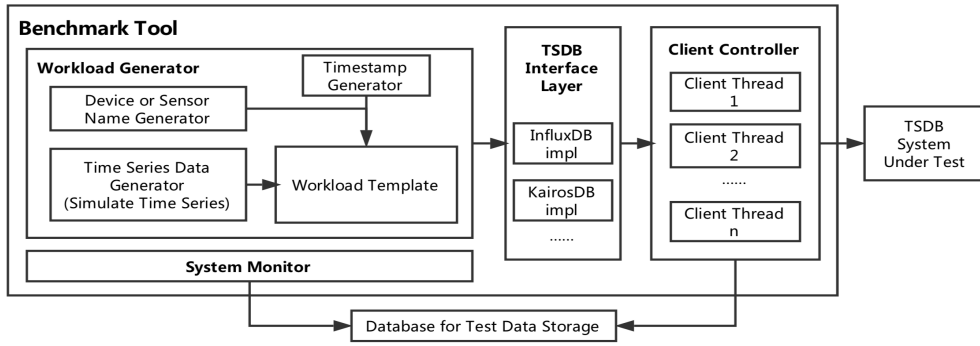


Figure 3: Benchmark Tool Architecture

Ingestion test. After the environment setup, the ingestion test will be performed based on the configured workload. The details of the ingestion test are in Section 5.3.

Query test. Since the query test is based on the data schema created in the ingestion test, the query test is usually performed after ingestion test. The details of query test are in Section 5.4.

Performance measurement. The performance metrics such as cost-time are measured in each ingestion and query operation. When the entire test is completed, a set of statistical performance metrics introduced in Section 4 will be calculated.

System monitoring. During the ingestion or query test process, the system resources will be measured as mentioned in Section 4. Through configuration, user can control the measuring frequency to fit different duration of test. Resources monitoring data can help us analysis different TSDB systems and provide another dimension of TSDB comparison.

Test data persistence. Most benchmark tools prepare test results by simply output them to console or log them into files, which needs users' extra effort to analyze the data. The test data includes configurations used in each test instance, all the performance measurements, and the system resource monitoring data. Our benchmark tool allow users to use a relational database, such as MySQL, to store all the test data while the test goes. By using a database to manage the test data, we can easily trace the test results, monitor test process and analyze the test data with SQL or better with a data visualization tool, like Tableau [4] or Grafana [1]. When performing long running test and there are many test instances running in parallel, this feature becomes absolutely necessary.

The above processes can be done automatically by our benchmark tool. Since there are many parameters can be configured, users can explore the performance impacts with these parameters. But the routine of manually configuring, initialization and launching test is tedious and exhausting. Our tool provide a simpler way to do a sequence of tests automatically by letting user edit a routine file, which defines what parameters should be altered before each test.

The modular design of the benchmark tool is shown in Fig. 3. The idea of modularization makes our tool scalable and extensible to add new features and support new TSDB systems. The tool contains 4 major parts: **workload generator**, **TSDB interface layer**, **client controller** and **system monitor**.

Workload generator: this module is responsible for generating ingestion or query SQLs or requests, which contains 4 sub-modules.

When generating a new ingestion SQL or request, the *workload generator* sub-module calls *timestamp generator* to get the next timestamp t_i and fills the template with t_i and the value generated by *series data generator*, which maintains a function of (t_i, s_i) . Currently, 5 types of functions are supported (as listed in Fig. 1, Section 3.1), and different sensors are assigned different value functions randomly from the above functions while the appearance ratio of these function obeys user's definition. Besides, the parameters of the functions are user-defined, so sensors may have the same type of value function while the parameters such as the period, the maximal value, and the offset varies from device to device.

When generating a query SQL or request, the *workload generator* generates time series list for a query, and then produces corresponding query clauses, such as the time/value filter and the aggregation function according to the query type. Then it fills the workload template with these values.

TSDB interface layer: to support more TSDB systems, all the data ingestion and query operations are abstracted in system-free interfaces. By implementing these interfaces, users can apply the benchmark on more kinds of TSDB systems.

Client controller: this module enables concurrent test through multiple client threads. The test client defines a higher level of test procedure, which consists of many basic operations that are implemented in the interface layer. This controller measures the cost-time of each operations and calculates the performance metrics when a test is complete. The performance metrics are stored into a relational database.

System monitor: to fulfill the duties of monitoring, this module is developed for automatically measuring system resources previously listed and storing the data into the same relational database as that of test results during the test procedures.

5.3 Ingestion Test

5.3.1 Define Ingestion Workload.

The ingestion workload can be configured by several parameters as shown in Table 1.

An example of ingestion workload parameters is listed in the rightmost column in Table 1 to illustrate the meaning of each parameter. Under the parameter set, the total number

Table 1: Main Ingestion Workload Parameters

Parameter name	Description	Example
GROUP_NUMBER	Total device group number, each group has several devices	2
DEVICE_NUMBER	Total device number	10
SENSOR_NUMBER	Sensor number per device	3
CLIENT_NUMBER	Concurrent client number	5
BATCH_SIZE	Record number per batch	100
EPOCH	Number of batch ingestion operations for each device	6
DATA_TYPE	Data type of ingestion data	DOUBLE
POINT_STEP	Milliseconds between two neighboring data points	5000
TIMESTAMP_GEN_MODE	Decide how timestamp is generated	0
IS_MUL_DEV_BATCH	Decide if one batch contains data of different devices	False
IS_RANDOM_INTERVAL	Decide whether add a noise to POINT_STEP	False
DISTRIBUTION_RATIO	Ratio of five data distribution types	1:1:1:1:1

of time series to generate is 30 (10×3), and the data from 10 devices are equally divided into 2 groups. The benchmark tool will use 5 client threads to send data ingestion requests to the TSDB service. Each client is bounded with a certain set of device evenly. It means client-thread-1 will only ingest the data of d_0 to d_1 and client-thread-2 d_2 to d_3 .

Data schema. Every TSDB has its own data schema and we need to map the benchmark parameters to TSDBs' specific ones. In the case of InfluxDB, the concept of the device group corresponds to its *measurement*, and the device to its *tag*. We map the concept of the sensor to its *field*, which means there are three fields (s_0 , s_1 and s_2) in each measurement. Therefore, the tag *device* take values from d_0 to d_4 in the measurement *group_0* and d_5 to d_9 in the measurement *group_1*. For another example, we use *tag* to distinguish devices from sensors and regard a device group as a *metric* when it comes to OpenTSDB, because its official document shows that the concept of metric is a group of time series rather than a single time series. In fact, we did some experiments to compare the query performance of regarding group as metric or tag, and the result shows the former is better.

Ordered Data Ingestion. Still refer to the example in Table 1. Each ingestion operation request (i.e. one batch) contains 100 records when BATCH_SIZE=100. Each record has a single device's all sensors data with the same timestamp, just like a row in relational database, which has three data points in the example.

One batch in epoch i for d_j is $B_{ij} = [R_{0ij}, R_{1ij}, \dots, R_{n_{ij}}]$ where $n = \text{BATCH_SIZE}$. The subscript i_j means batch B_{ij} belongs to epoch i and device d_j , which means the data of one batch all belong to a single device. A record $R_{kij}(\text{timestamp}, s_0, s_1, \dots, s_m) = (t_{kij}, f_0(t_{kij}), f_1(t_{kij}), \dots, f_m(t_{kij}))$ where $m = \text{SENSOR_NUMBER}$, $t_{kij} = (i \times \text{BATCH_SIZE} + k) \times \text{POINT_STEP}$ and f_m is the data distribution function assigned to s_m . When all devices complete one batch ingestion, it means epoch i is done, and then the next epoch $i + 1$ begins. For example, the first batch (i.e., epoch 0) for d_j is $B_{0j} = [R_{00j}, R_{10j}, \dots, R_{990j}]$ where $R_{00j} = (0, 4.1, 6.4, 5.7)$, $R_{10j} = (5000, 8.2, 5.0, 5.8)$, $\dots$, $R_{990j} = (495000, 3.8, 3.2, 9.7)$. Therefore, the total number of data points in one batch equals to $\text{SENSOR_NUMBER} \times \text{BATCH_SIZE}$. Since IS_RANDOM_INTERVAL is set to false and TIMESTAMP_GEN_MODE is 0 (meaning no out-of-order data), the timestamp of each sensor increases evenly. The whole ingestion test is completed when six epochs are done.

When the insertion test is completed, each sensor/series

has $\text{BATCH_SIZE} \times \text{EPOCH}$ data points and the total number of data points is $\text{BATCH_SIZE} \times \text{EPOCH} \times \text{SENSOR_NUMBER} \times \text{DEVICE_NUMBER}$.

Write operations are often performed in batches in actual applications. We use parameter BATCH_SIZE to specify batch-write operations. When BATCH_SIZE = 1, the benchmark tool writes data point by point, which is equivalent to the write mode of many other test tools.

Out-of-order Data Ingestion. The timestamp generating algorithm can be set by the TIMESTAMP_GEN_MODE parameter. There are three types of out-of-order modes, among which Poisson-distribution out-of-order mode is most commonly used. In this section we describe the specific implementation and mechanism to control the proportion P of out-of-order data.

DEFINITION 1. A time series ingestion workload S is an ordered array of time-data tuple $T_i = (t_i, d_i)$

$$S = [(t_0, d_0), (t_1, d_1), \dots, (t_i, d_i), \dots, (t_n, d_n)] \quad (1)$$

where the indicator i is in the order the data arrive at TSDB, which means for any $i < j$, time-data tuple T_i is sent to TSDB before T_j , or is previous to T_j if in the same batch.

The out-of-order data is the time-data tuple T_i which satisfy:

$$t_i < \max\{t_j | 0 \leq j \leq i, j \in \mathbb{N}\} \quad (2)$$

To calculate the timestamp of the next data point t_i , the benchmark tool maintains a maximum timestamp CMT (Current Max Timestamp) of the currently written data. t_i may be smaller than the current maximum timestamp CMT by probability P , and how much smaller is decided by a random variable X , which obeys Poisson distribution. Otherwise, t_i increases one step size, which happens by probability $1 - P$. To formalize these descriptions, we give the following formula:

$$t_i = \begin{cases} CMT - \Delta T, & P \\ CMT + \text{POINT_STEP}, & 1 - P \end{cases} \quad (3)$$

where

$$CMT = \max\{t_j | 0 \leq j \leq i, j \in \mathbb{N}\} \quad (4)$$

$$\Delta T = \text{POINT_STEP} \times (X + 1) \quad (5)$$

$$X \sim \text{Poisson}(\lambda) \quad (6)$$

$$\text{i.e. } P(X = k) = \frac{\lambda^k}{k!} e^{-\lambda}, \quad k \in \mathbb{N} \quad (7)$$

Table 2: Main Query Workload Parameters

Parameter name	Description	Example
QUERY_TYPE	Query workload type	10
QUERY_SENSOR_NUM	Number of sensors involved in each query	2
QUERY_DEVICE_NUM	Number of devices involved in each query	2
QUERY_AGG_FUN	Aggregation functions used by aggregation query	max
CLIENT_NUMBER	Query client number	2
EPOCH	Query requests number of each client	100
QUERY_SPAN	Time filter span (ms)	600000
QUERY_VAL_FILTER	Query value filter	> 0
TIME_INTERVAL	Interval of GROUP-BY-time query (ms)	60000

5.4 Query Test

5.4.1 Define Query Workload

Query load can be generated by randomly changing the parameter value in query filters for a particular type of query. By default, query test is based on the data generated by ingestion test. Therefore, the ingestion workload parameters related to data schema also affect query test. Apart from that, the main relevant parameters for query test are shown in Table 2:

Taking *Q10-Group by time range query* as an example, the parameters are set as Table 2 and the data set is generated by an ingestion test which uses configurations in Table 1. The benchmark tool will use two client threads to send query requests to the target TSDB service.

Here, we use InfluxDB as an example to illustrate the actual query workload corresponding to these parameters. For other TSDBs, the query workload will be converted into equivalent operations. The queries are of the following format:

```
SELECT max(s_0), max(s_1)
FROM group_0, group_1
WHERE ( device = 'd_3' OR device = 'd_8')
AND time >= 2010-01-01 12:00:00 AND time <= 2010-01-01
12:10:00
GROUP BY time(60000ms)
```

where the sensors, devices and time range in the WHERE clause of each query are randomly selected.

This query means that each result point is the maximum of data in 1 minute span interval, which evenly divides the time range. Since the time range in query is 10 minutes, the query should return 10 aggregation result points for each data series. In this case, there are four series queried (*s_0* and *s_1* of *d_3* in *group_0* plus *s_0* and *s_1* of *d_8* in *group_1*), then the query should return 40 result points in total.

5.5 Experimental Setup

To reduce system complexity and random noise effect, we performed experiments on two server-class machines (two Intel Xeon CPU E5-2697 v4 @ 2.30GHz processors, 256-GB of memory, 10 disk RAID-5 array and 10 gigabit ethernet, the Operating System is Ubuntu 16.04.2 LTS 64-bit). One machine is used to run the TSDB service and the other the benchmark tool. Therefore, all tests are based on single-node TSDB installation. We monitor the system resources from client machine and give the client enough system resource so that the client machine wouldn't be a bottleneck.

In this paper, we benchmark four TSDB systems: InfluxDB 1.5.1, OpenTSDB 2.3.1 based on Hbase 1.2.8, KairosDB 1.2.1 based on Cassandra 3.11.3 and TimescaleDB 1.0.0 based on PostgreSQL 10.5. The configuration of each TSDB is

tuned as much as we know to release its potential. We allocate enough and equal RAM or JVM space to each TSDB so that memory space wouldn't be a limitation. In particular, we set cache-max-memory-size and max-series-per-database of InfluxDB to unlimited. For OpenTSDB, we configured some parameters like `tsd.http.request.enable_chunked`, `tsd.http.request.max_chunk` and `tsd.storage.fix_duplicates` to enable large batch and out-of-order data ingestion. For KairosDB [14], the parameter `read_repair_chance` of Cassandra is set to 0.1 as official document instructed. Besides, we use PG Tune [3] to calculate the general optimized configuration of PostgreSQL for TimescaleDB.

5.6 Data Ingestion

As introduced in Section 3.1 and Section 5.3, IoTDB-Benchmark provides many user configurable parameters to simulate real industrial time series data ingestion workloads. Our benchmark framework/tool enables users to explore the performance of TSDB systems and help them understand the relationship between performance and parameters, which is significant for system tuning. For ingestion test, we show several experiments that cover the five user-defined aspects mentioned in Section 5.3, to show the tip of the iceberg.

Concurrent client number: we compared the ingestion performance with different client numbers by configuring `CLIENT_NUMBER` as shown in Fig.4(a). Except the client number, all TSDBs use the same configurations: **1000 devices in total and each device has 10 sensors of DOUBLE type. Each sensor/time-series will have 10000 data points (with BATCH_SIZE=100, EPOCH=100)** to be ingested and `DISTRIBUTION_RATIO=1:1:1:1:1`. The configuration is regarded as a standard for other ingestion experiments.

The result shows that, **with the client number increasing, the throughput of TimescaleDB and InfluxDB are growing while OpenTSDB and KairosDB are not.** Especially for OpenTSDB, when the client number is bigger than 30, its ingestion performance drops dramatically so that the test can't even finish in the endurable time. Therefore, we only present the results of **OpenTSDB when client number is fewer than 30.**

The number of time series: the time series number equals to `DEVICE_NUMBER × SENSOR_NUMBER`, therefore we set `SENSOR_NUMBER=100` and change `DEVICE_NUMBER` to compare the ingestion performance under different time series number as shown in Fig.4(b). In this experiment group, we use 20 clients and enlarge the time series number to millions. We set `BATCH_SIZE=10` to make the data points number per batch the same with previous experiments in Fig.4(a) since `SENSOR_NUMBER=100` and keep other parameters the same.

Comparing Fig.4(b) and Fig.4(a) we can see that with the

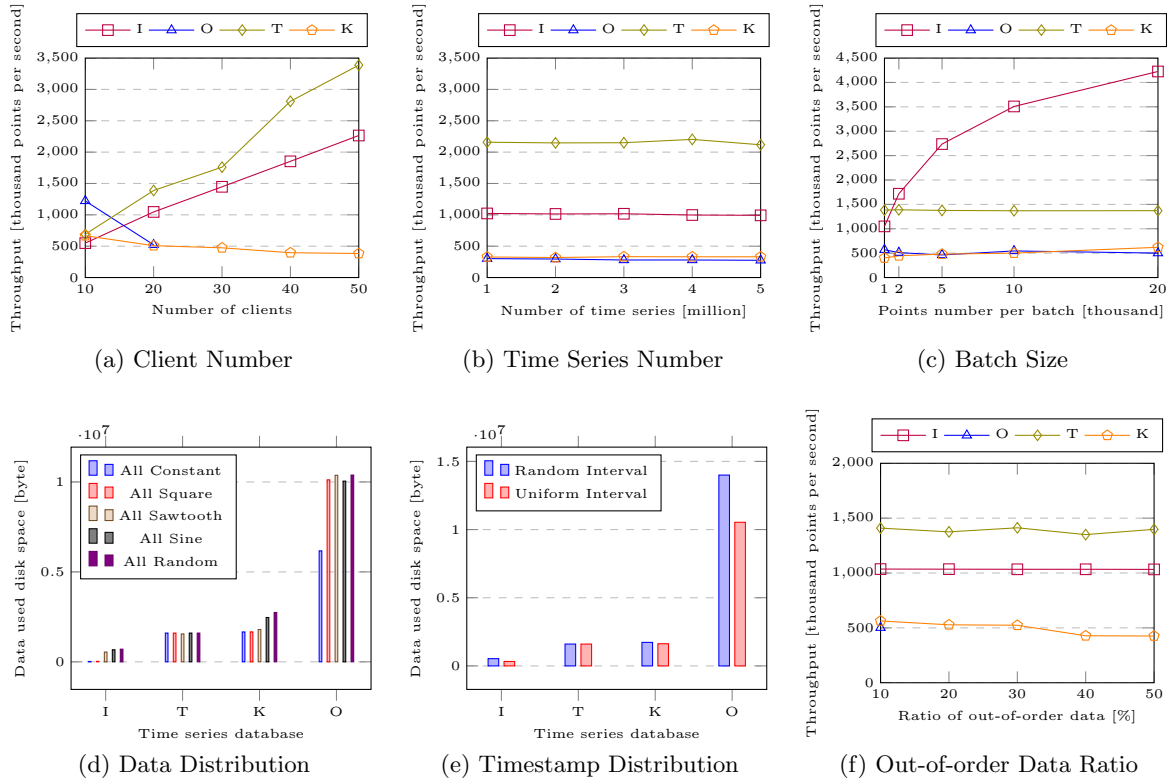


Figure 4: Ingestion Experiments. **Respectively**
(Throughput unit: 1000 points/second)

time series number increasing, the throughput of InfluxDB is almost the same. **TimescaleDB has even better performance** when the sensors number in each record increases from 10 to 100, but the throughput of KairosDB and OpenTSDB dropped. From Fig.4(b), we can see that enlarging the series number has little impact on ingestion performance.

Batch size: in Fig. 4(c) we compare the throughput with different BATCH_SIZE. We use 20 clients and other parameters are the same as those in Fig. 4(a).

The result shows **the throughput of InfluxDB grows quickly when the batch size increasing, which has limitation of 7 million points per second (not shown). While the throughput of other TSDBs have insignificant changes.**

The data distribution: as mentioned in Section 4 our performance metric also involves disk space consumption, which is especially related with time series data distribution type. We set different DATA_RATIO (e.g. 1:0:0:0 for all constant) to compare the performance when ingesting different types of time series data. Other parameters are the same.

Results in Fig. 4(d) show the disk usage efficiency of the four TSDBs, where $\text{InfluxDB} > \text{TimescaleDB} \geq \text{KairosDB} > \text{OpenTSDB}$. For different distribution types, the disk space consumptions are also different, but the difference of throughput is insignificant (not shown).

The timestamp distribution: to set IS_RANDOM_INTERVAL to True can make the time interval of neighboring points irregular. Other parameters follow previous configurations. We compare the disk space consumption of the two scenarios as shown in Fig. 4(e) and the result shows that the disk space consumption with uniform timestamp interval data is less than that of random timestamp interval data. Moreover, random time interval has little effect on ingestion throughput (not shown).

Out-of-order Ingestion: the data of above ingestion tests are all in order of timestamp and we then alter to out-of-order data ingestion workload. We set TIMESTAMP_GEN_MODE=3, which is the Poisson-distribution out-of-order and change the ratio of out-of-order data (a parameter that not shown in Table 1) as shown in Fig. 4(f). Other parameters follow previous configurations.

The result shows that for InfluxDB, TimescaleDB and KairosDB, the throughput may slightly slow down when the out-of-order data ratio increases. While for OpenTSDB the out-of-order data leads to sharp atrophy of throughput, therefore we only present result of 10% out-of-order data.

5.7 Query

As introduced in Section 3.2, our benchmark supports 10 types of query. Because it is tediously long to illustrate all of them, we choose the typical four types of queries that cover the four basic query types in real scenarios listed in Section 2, as the query workload for benchmarking the same four TSDBs.

The four types are: Q1-Exact point query, Q2-Time range query, Q6-Aggregation query with time filter and Q10-Group by time range query. Fig. 5 shows the results of the four TSDBs's average cost-time per query under different query workloads, in which Q1 is corresponding to sub-figure (a), Q2 is (b) and (c), Q6 is (d) and (e), and Q10 is (f). Every query workload consists of 100 query requests and only one client.

Data Set: all these query tests are based on the data generated by a ingestion test, which contains 10 device groups.

Each device group has 100 devices and each device has 10 sensors, therefore 10000 ($10 \times 100 \times 10$) time-series in total. Each sensor (i.e., time-series) contains 10000 data points with uniform time interval of 5 seconds.

5.8 System Resource Monitoring

Q1-Exact Point Query: when we compare the exact point query performance of series number changes in each query, we fix QUERY_SENSOR_NUM=10 and set QUERY_DEVICE_NUM = 2,4,6,8,10 respectively. We use the routine way introduced in Section 5.1 to execute this sequence of tests automatically and collect the results in Fig. 5(a). The result shows that the TimescaleDB is the fastest, and the next is InfluxDB and OpenTSDB, which are close to TimescaleDB, while KairosDB is the worst and about 35 times slower than TimescaleDB. This is because TimescaleDB is based on a relational database, PostgreSQL and uses time as the primary key, which is specially indexed. Besides, the effect of series number in each query is not significant.

Q2-Time Range Query: in Fig. 5(b), we compare the time range query performance. When the time range changes in each query, it means the number of each query's result points changes. We can see that the performance of InfluxDB and KairosDB are the best and the change of the time range has little influence on them. For TimescaleDB and OpenTSDB, the cost-time increased significantly with number result points increasing. In Fig. 5(c), we set the time range to 50000000ms, which covers all timestamps, and change the series number in each query. We discover that the performance of InfluxDB is sensitive to series number comparing with other TSDBs.

Q6-Aggregation Query: we used COUNT and MAX as aggregation functions applied in Aggregation Query experiments. Each function is applied on different series number in each query as shown in Fig. 5(d) and (e). The results show that these two aggregation functions have almost the same performance for every TSDB. Besides, InfluxDB is the fastest, followed by OpenTSDB, TimescaleDB and KairosDB.

Q10-Group By Time Range Query: we use MAX as the aggregation function in the group by time range query and set QUERY_SPAN=50000000, which covers the whole series. By configuring TIME_INTERVAL, we compared the performance of different time range span as shown in Fig. 5(f). For example, the *group by time range span* is 1% means TIME_INTERVAL is $1\% \times 50000000ms = 500000ms$. Therefore, each query will get $\frac{1}{1\%} = 100$ aggregated points in the query result. We can see that the bigger the aggregation granularity, the better the performance, because the bigger aggregation granularity results in fewer result points.

From the above query benchmark results, we can see that InfluxDB is leading the TSDBs since its performance in most query workloads are far better than the others. The query performance of OpenTSDB and TimescaleDB are close under many circumstances, while KairosDB is relatively weak, compared with other TSDBs, except the time range query. Our benchmark tool can not only collect test results, but also monitor the system resource consumption during the test process. We visualize the monitoring data listed in Section 4 as shown in Fig. 6, which monitors InfluxDB in Fig. 4(b) when the series number is 1 million. We can see that the disk usage keeps on a high level while the CPU usage is about 6%. It takes about one hour to run the whole test.

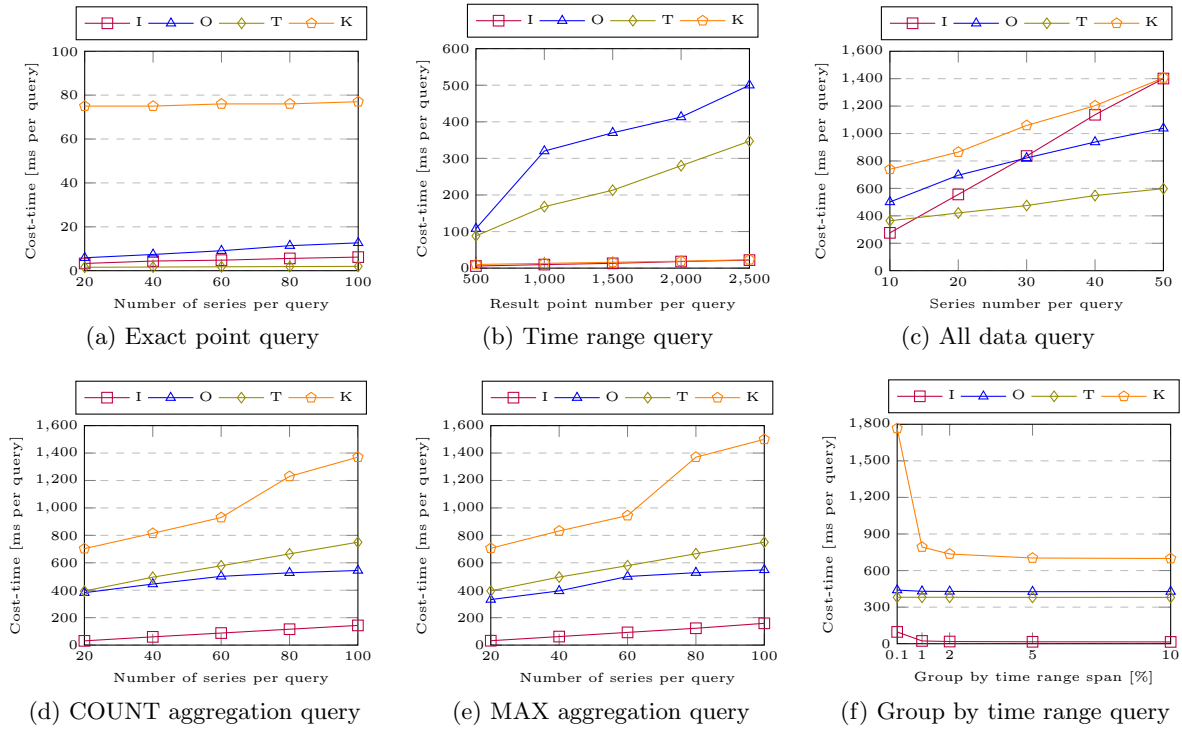


Figure 5: Four TSDBs's Average Cost-time of Different Query Workload Types

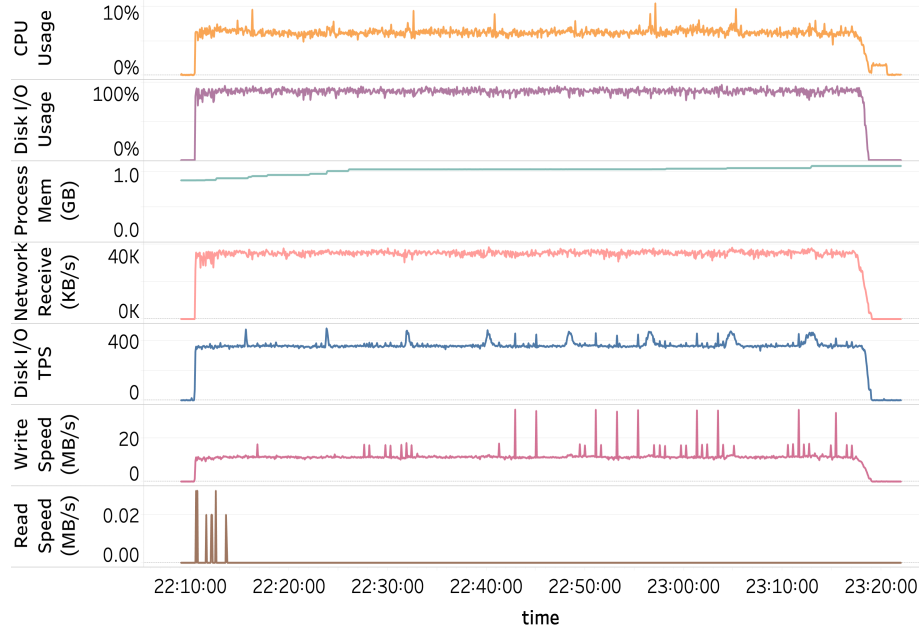


Figure 6: System Resource Consumption Monitoring During InfluxDB Ingestion Test

Due to limited space, the system resource consumptions of other TSDBs are not demonstrated.

6 Related Work

Benchmark and Tool. Besides TPC benchmark and YCSB, there are other benchmarks and corresponding tools in the big data arena for our reference. YCSB-TS [7] is a branch of YCSB that measures the performance of a TSDB. Andreas Bader [8] used YCSB-TS to compare different TSDB systems. Since YCSB-TS is based on YCSB, it inherits some of the shortcomings of YCSB that have been encountered before. Ghazal [13], et al, developed Bigbench to benchmark DBMS and its design ideas and extensibility are of research value. Chowdhury [11] extends Bigbench to test Hadoop. Hiben [16] is a benchmark tool developed by Intel for Hadoop. With the launch of Spark, BigDataBench, a big data benchmark tool, developed by UC Berkeley AMPLab Lab, was used to test big data systems, such as Spark. There are also other benchmark/tools that come with or associated to the database. For example *influxdb-comparisons* is a comparison test tool written in Go language used by InfluxDB to compare InfluxDB with other NoSQL systems. However these benchmarks serve for specified products, which may lack justification for the compared systems. **Time series Database.** The TSDBs we compared in this paper are considered as typical TSDBs because they covered 3 categories [9] of TSDB: (1) TSDB with no requirement on any DBMS; (2) TSDB with a requirement on NoSQL DBMS; (3) TSDB based on or modified from a relational database. InfluxDB, written in Go language, is one of the most popular time series data management solutions. It has its own storage engine with TSM-Tree [18], which is an optimization of LSM-Tree [19]. OpenTSDB is a distributed and scalable TSDB based on HBase [15]. Similar to InfluxDB, it uses tag to mark different series. KairosDB is forked from OpenTSDB, but is mostly based on Cassandra [10] storage while it also supports in memory storage called H2. Since Cassandra's rows are wider than HBase, KairosDB's Cassandra has a default row size of 3 weeks, while OpenTSDB's HBase is 1 hour. TimescaleDB is a TSDB based on relational database PostgreSQL [26], which still uses data schema of traditional relational database, but it is especially optimized for time-series data. It supports PostgreSQL's full SQL, which makes TimescaleDB inherit advantages of relational databases while meeting the needs of time-series data management.

7 Conclusion

In this paper, we present IoTDB-Benchmark for evaluating time series databases. The benchmark considers industrial IoT scenarios as the typical application of time series database and provides various parameters to simulate different scenarios and corresponding workloads in the real world. Correspondingly, we develop a benchmark tool to interpret those parameters, to conduct the benchmark testing, and to collect both test result data and system resource consumption data. We apply IoTDB-Benchmark and conduct several groups of experiments, including the ingestion test and query test on four popular TSDBs, InfluxDB, OpenTSDB, KairosDB and TimescaleDB. The results show the insights on them for developers and IT managers. The contributions and novel features of our work are as follow, first, we present a TSDB benchmark, which is especially for TSDB

and designed for various application scenarios. Second, our benchmark takes system resource consumption metrics into consideration for recording, which is crucial for analyzing TSDB systems. Third, we apply relational database to manage the test data like performance metrics and corresponding configurations, which enable users to trace test results conveniently and conduct further analysis. With these features, IoTDB-Benchmark is able to provide benchmark for both development and research purpose. We will expand it for more TSDBs and add more workloads variations targeting on more complex scenarios in the future.

8 References

- [1] Grafana. <https://grafana.com>.
- [2] influxdb-comparisons. <https://github.com/influxdata/influxdb-comparisons>.
- [3] Pgtune. <https://pgtune.leopard.in.ua>.
- [4] Tableau. <https://www.tableau.com>.
- [5] Tpc. <http://www.tpc.org>.
- [6] Tpcx-iot. <http://www.tpc.org/tpcx-iot/default.asp>.
- [7] Ycsb-ts. <http://tsdbbench.github.io/YCSB-TS>.
- [8] A. Bader. Comparison of time series databases. Master's thesis, 2016.
- [9] A. Bader, O. Kopp, and M. Falkenthal. Survey and comparison of open source time series databases. *BTW 2017-Workshopband*, 2017.
- [10] A. Cassandra. Apache cassandra. *Website. Available online at <http://planetcassandra.org/what-is-apache-cassandra>*, page 13, 2014.
- [11] B. Chowdhury, T. Rabl, Saadatpanah, and H.-A. et.al. A bigbench implementation in the hadoop ecosystem. In *Workshop on Big Data Benchmarks*, pages 3–18. Springer, 2013.
- [12] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears. Benchmarking cloud serving systems with ycsb. In *Proceedings of the 1st ACM symposium on Cloud computing*, pages 143–154. ACM, 2010.
- [13] A. Ghazal, T. Rabl, M. Hu, Raab, and et.al. Bigbench: towards an industry standard benchmark for big data analytics. In *SIGMOD*, pages 1197–1208. ACM, 2013.
- [14] B. Hawkins. Kairos db: Fast time series database on cassandra, 2017.
- [15] A. HBase. Apache hbase, 2013.
- [16] S. Huang, J. Huang, J. Dai, T. Xie, and B. Huang. The hiben benchmark suite: Characterization of the mapreduce-based data analysis. In *ICDEW, 2010 IEEE 26th International Conference on*, pages 41–51. IEEE, 2010.
- [17] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz. Aries: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM TODS*, 17(1):94–162, 1992.
- [18] S. N. Z. Naqvi, S. Yfantidou, and E. Zimányi. Time series databases and influxdb. 2017.
- [19] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil. The log-structured merge-tree (lsm-tree). *Acta Informatica*, 33(4):351–385, 1996.
- [20] T. Pelkonen, S. Franklin, J. Teller, P. Cavallaro,

- Q. Huang, J. Meza, and K. Veeraraghavan. Gorilla: A fast, scalable, in-memory time series database. *Proceedings of the VLDB Endowment*, 8(12):1816–1827, 2015.
- [21] D. Pountain. Run-length encoding. *Byte*, 12(6):317–319, 1987.
- [22] I.-O. S. T. Series. Metrics, and analytics database. *Website Httpinfluxdb Com*, 2015.
- [23] B. Sigoure. Opentsdb scalable time series database (tsdb). *Stumble Upon*, 2012.
- [24] E. Štefancová. Evaluation of the timescaledb postgresql time series extension. 2018.
- [25] M. Strohbach, H. Ziekow, V. Gazis, and N. Akiva. Towards a big data analytics framework for iot and smart city applications. In *Modeling and processing for next-generation big-data technologies*, pages 257–282. Springer, 2015.
- [26] D. Vohra. Using postgresql database. In *Kubernetes Microservices with Docker*, pages 115–139. Springer, 2016.